

Survival Analysis — The Friendly Guide

Intuition first, formulas optional
NSUCEC Datathon · Team Cybernauts

How to read this guide

Every topic starts with a **plain-English story** (usually about customers leaving an app, or light bulbs burning out). The **purple boxes** hold the actual formula — read them only if you want the math; you can understand everything without them. The grey boxes connect each idea back to the churn project you already built.

Contents

1	The One Big Idea	1
2	The Two Ways to Look at Survival	1
2.1	The survival curve $S(t)$ — “what % are still alive?”	1
2.2	The hazard $h(t)$ — “how dangerous is right now?”	2
3	Kaplan–Meier: Drawing the Curve from Messy Data	2
3.1	Comparing two groups: the log-rank test	3
4	The Cox Model: Many Factors at Once	3
4.1	The one catch: the “proportional hazards” assumption	4
5	Weibull: Assuming a Shape for the Risk	4
6	The Machine-Learning Versions	5
7	Grading a Survival Model	5
7.1	C-index — “does it rank people correctly?”	5
7.2	Brier score — “are the probabilities accurate?”	6
8	The Two Python Toolboxes	6
9	30-Second Recap (read this the morning of)	6
10	One-Page Cheat Sheet	7
11	Likely Questions (with plain answers)	7

1 The One Big Idea

Survival analysis answers a single question: “**how long until something happens?**” — where the “something” (called the **event**) is usually bad: a person dies, a machine breaks, a customer churns.

Here is the twist that makes it special. Imagine you switch on **100 light bulbs** and watch them. Some burn out and you write down exactly when. But after 6 months you stop watching, and **30 bulbs are still glowing**. You do *not* know how long those will last — only that it is *more than 6 months*.

The naive move is to throw those 30 away (“incomplete data”). That is a terrible idea — *those are your best bulbs!* Ignoring them makes your product look worse than it really is.

The whole point of the field

Survival analysis is simply **the correct way to use “still alive when I stopped watching” information**. That half-known data point has a name — **censoring** — and handling it properly is the entire reason this toolbox exists.

How this maps to your FictiPay churn project

A customer = a light bulb. Churning = burning out. A customer who is *still active* at your April 1 cutoff = **a bulb still glowing = censored**. You know they “survived” (kept transacting) at least that long — you just do not know *when* they will finally quit.

2 The Two Ways to Look at Survival

There are two questions you can ask about your bulbs (or customers), and they are just two views of the *same* story.

2.1 The survival curve $S(t)$ — “what % are still alive?”

Starts at 100%, only ever goes *down*. At month 3, maybe 80% are still glowing; at month 6, 60%. That downward curve is the **survival function**. Simple.

2.2 The hazard $h(t)$ — “how dangerous is right now?”

This is the subtle one. The hazard is the **risk rate for the survivors only**: of those still alive *at this very moment*, what is the chance they fail in the next instant?

A bulb that has already lasted 5 years might be *more* likely to pop soon — it is worn out. A brand-new customer might be fragile too (never really got hooked). The hazard captures “how risky is *this particular moment*, given you have made it this far.”

The mental picture

Think of the **hazard** as a *danger meter* ticking moment by moment, and the **survival curve** as the *scoreboard*. The danger meter is what *drives* the scoreboard down. High hazard now $\Rightarrow$ the survival curve is about to drop steeply.

If you want the formula

Four ways to describe the same thing, all interchangeable:

$$S(t) = P(T > t) \text{ (still alive),} \quad h(t) = \frac{f(t)}{S(t)} \text{ (risk rate now),}$$

$$H(t) = \int_0^t h(u) du \text{ (risk piled up so far),} \quad \boxed{S(t) = e^{-H(t)}}.$$

That last boxed identity is the bridge: total accumulated danger H determines how many survive.

How this maps to your FictiPay churn project

Your best features — `recency_days`, `gap_z`, `p_silent30_poisson` — are basically **hazard readings**. A customer silent for 40 days when they normally transact daily has a *sky-high* hazard right now (danger meter pegged). That is exactly why those features dominated your model.

3 Kaplan–Meier: Drawing the Curve from Messy Data

Kaplan–Meier (KM) is just the method for *drawing* that “% still alive over time” curve when some data is censored. You do not need the formula — you need the picture.

```
% still alive
100% | #####
      | ##### <- flat stretch: nobody churned here
80%  | ### (some may be CENSORED here -- they
      | ##### just leave the pool quietly,
60%  | ##### they do NOT cause a drop)
      | ###
40%  | ## <- each STEP DOWN = one real churn
      +-----> time
```

It is a **staircase going down**. Every step down = one real event (a churn). Flat stretches = nobody churned. The clever part: when someone is **censored** (still active when you stop watching), they do *not* cause a step down — they just **quietly leave the pool** so they are not counted in later steps. That is how KM honestly uses your “still-glowing bulbs.”

Two things people always ask about

Median survival = the time where the staircase crosses 50% (the “typical” lifespan). If the curve never reaches 50% (lots of censoring), the median is simply “not reached” — a perfectly valid answer. **Why no drop at a censoring time?** Because a censored person never had the event; they only stopped being watched.

If you want the formula

At each event time t_i : d_i = how many failed, n_i = how many were still at risk just before.

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right).$$

Each bracket is “fraction who survived this instant”; multiply them along the staircase.

3.1 Comparing two groups: the log-rank test

Say you split customers into **high-balance vs low-balance**, draw a KM staircase for each, and the high-balance one stays higher (they churn less). Is that a *real* difference, or just luck?

In plain words

The **log-rank test** answers exactly that. It tallies “how many churned in each group, and when” versus “what we would expect if the two groups were secretly identical.” A big mismatch $\Rightarrow$ the groups are genuinely different. It gives you a **yes/no with a confidence level (p-value)** — but *not* “by how much.” For “by how much,” you need the Cox model (next).

Easy to mix up

Kaplan–Meier and the log-rank test only compare **groups** (categories). The moment you want to handle a continuous number (balance, age, `gap_z`) or several factors at once, you must move to a *regression* model — Cox or Weibull.

4 The Cox Model: Many Factors at Once

KM compares groups. But real customers have **lots** of traits at the same time — age, balance, region, recency, transaction count. You cannot draw a separate curve for every combination.

The **Cox proportional hazards model** fixes this. Its whole idea in one sentence:

The Cox idea

*Everyone shares one baseline “danger-over-time” shape, and your personal traits **multiply** that baseline up or down.*

The output for each trait is a **hazard ratio (HR)** — the multiplier:

- **HR** = 2 $\Rightarrow$ that trait *doubles* the churn rate (a risk factor).
- **HR** = 0.5 $\Rightarrow$ *halves* it (protective).
- **HR** = 1 $\Rightarrow$ no effect.

Example: “each extra week of silence has **HR** = 1.4” means every quiet week **multiplies** churn risk by 1.4 $\times$. That is a clean, boardroom-friendly sentence — far more useful than a raw probability.

If you want the formula

$$h(t \mid \mathbf{x}) = \underbrace{h_0(t)}_{\text{shared baseline}} \times \underbrace{e^{\beta_1 x_1 + \dots + \beta_p x_p}}_{\text{your personal multiplier}}, \quad \text{HR}_j = e^{\beta_j}.$$

“Semi-parametric” = it never has to guess the baseline shape $h_0(t)$; that cancels out in the math.

4.1 The one catch: the “proportional hazards” assumption

In plain words

Cox **assumes your multiplier stays the same over time**. If silence always multiplies risk by 1.4 $\times$ — whether the customer is brand new or two years old — great, Cox is valid. But if a trait matters a lot *early* and fades *later*, Cox is wrong and you must check it.

How to check it (know one or two):

- **The parallel-lines test** (log–log plot): transform each group’s curve a certain way and look. **Stay parallel** $\Rightarrow$ assumption holds. **Cross or fan apart** $\Rightarrow$ broken.
- **Schoenfeld residuals**: a built-in statistical test (`cph.check_assumptions` in `lifelines`) that flags any trait whose effect drifts with time.

If it is broken: split that trait into separate baselines (“stratify”), let its effect change over time, or switch to a Weibull / random-survival-forest model that does not require the assumption.

Exam tip

Cox tells you *relative* risk (“1.4× more likely”), not *when*. If the question is “*how many days sooner* will they churn?”, that is the parametric / AFT model’s job.

5 Weibull: Assuming a Shape for the Risk

Cox leaves the baseline danger-meter shape a *mystery* (just traced from the data). A **parametric** model instead *commits* to a known mathematical shape. **Weibull** is the famous flexible one, and it has a single “**shape knob**,” called k , that is the most-tested fact in this whole topic:

The Weibull shape knob k

- $k > 1$ — **danger keeps rising**. Things wear out (old machines, aging bulbs).
- $k = 1$ — **danger is flat**. Same risk forever, no matter how long it has been (purely random failures). This is the plain “exponential” case.
- $k < 1$ — **danger falls over time**. Survive the shaky beginning and you are golden. **This is classic for customers:** many churn in the first few weeks, but whoever sticks around becomes loyal for the long haul.

Why use Weibull instead of Cox? Three reasons:

1. **Predict the future.** Cox cannot say anything past your last observed time; a Weibull curve keeps going, so you can forecast.
2. **Smooth curves** and the ability to *simulate* customer lifetimes.
3. **The shape itself is the insight** (“our churn is $k < 1 \Rightarrow$ fix onboarding, that is where we bleed people”).

If you want the formula

$$S(t) = e^{-(t/\lambda)^k}, \quad h(t) = \frac{k}{\lambda} \left(\frac{t}{\lambda} \right)^{k-1}.$$

k = shape (rising/flat/falling), λ = scale (overall “stretch” of the timeline). The related **AFT** view writes time directly: a trait’s “time ratio” e^β says the event happens that many times *sooner* or *later*.

6 The Machine-Learning Versions

When relationships get complicated (lots of interacting features, no clean assumptions), you bring in machine learning — the *same* model families you already used, just taught to handle *time + censoring*.

- **Random Survival Forest (RSF)** — a random forest where each tree predicts time-to-event and knows how to handle censored customers. It splits the data to separate “high-risk soon” from “low-risk” groups as sharply as possible. No proportional-hazards assumption; finds non-linear patterns automatically.
- **XGBoost AFT** — gradient boosting (your XGBoost / CatBoost world) that predicts the *actual time until churn* and properly uses “still active” rows. You feed each customer a *range* for their event time: for a still-active customer, “somewhere after today” (lower bound = today, upper bound = infinity).

In plain words

Same machinery you already know from the churn round. The only new trick: these output a **“when”** (a time or a survival curve), not just a yes/no.

How this maps to your FictiPay churn project

You solved churn as a *yes/no at a fixed date* (“any transaction in April?”), which was the right call because the target is defined on a fixed window. But underneath, it is a survival problem: time-to-next-transaction, censored at the cutoff. If a judge asks *“why not survival analysis?”*, the crisp answer is: *“our label is a fixed-time yes/no, so classification directly optimises the scored metric. Survival would be the right tool if we needed churn **timing** or a churn-by-any-future-date curve.”*

7 Grading a Survival Model

7.1 C-index — “does it rank people correctly?”

In plain words

The **C-index** is the survival version of **AUC**. Take any two customers: did the model correctly guess which one churns sooner? The fraction it gets right is the C-index. **0.5 = coin flip, 1.0 = perfect**. It measures *ranking / ordering*.

7.2 Brier score — “are the probabilities accurate?”

In plain words

The **Brier score** checks whether the actual predicted *probabilities* are right, not just the ranking. **Lower = better**. It catches a model that ranks customers fine but outputs silly probability numbers (e.g. says “90% sure” and is wrong half the time).

Exam tip

Report **both**: a model can rank well (good C-index) but be badly calibrated (bad Brier), or the reverse. C-index = *discrimination*; Brier = *calibration + discrimination*. This contrast is a favourite exam question.

8 The Two Python Toolboxes

- **lifelines** — the friendly one. Best for Kaplan–Meier curves, the Cox model, and clear interpretation/plots.
- **scikit-survival (sksurv)** — sklearn-style. Best for the ML models (RSF) and the scoring metrics (C-index, Brier).

The bare-minimum code you should recognise:

```
# --- lifelines: Kaplan-Meier curve ---
from lifelines import KaplanMeierFitter, CoxPHFitter
kmf = KaplanMeierFitter()
kmf.fit(durations=df["time"], event_observed=df["event"])
kmf.plot_survival_function()           # the staircase
kmf.median_survival_time_              # the typical lifespan
```

```

# ---- lifelines: compare two groups ----
from lifelines.statistics import logrank_test
res = logrank_test(a["time"], b["time"], a["event"], b["event"])
print(res.p_value)                # small p -> groups really differ

# ---- lifelines: Cox model (hazard ratios) ----
cph = CoxPHFitter().fit(df, duration_col="time", event_col="event")
cph.print_summary()                # look at exp(coef) = the HR
cph.check_assumptions(df)         # is the PH assumption ok?

# ---- scikit-survival: Random Survival Forest + scoring ----
from sksurv.util import Surv
from sksurv.ensemble import RandomSurvivalForest
from sksurv.metrics import concordance_index_censored
y = Surv.from_arrays(event=df["event"].astype(bool), time=df["time"])
rsf = RandomSurvivalForest(n_estimators=300, n_jobs=-1).fit(X, y)
risk = rsf.predict(X_test)
c = concordance_index_censored(y_test["event"], y_test["time"], risk)[0]

```

9 30-Second Recap (read this the morning of)

In plain words

Watch a bunch of customers. Some churn (you see *when*); some are **still active when you stop watching** (censored — the whole point of the field). **Kaplan–Meier** draws the survival staircase. **Log-rank** tests whether two groups' staircases really differ. **Cox** says how each trait *multiplies* churn risk (the **hazard ratio**). **Weibull** assumes a shape and tells you whether risk *rises, stays flat, or falls* over a customer's life (the shape knob k). **RSF** / **XGBoost-AFT** are the machine-learning versions. **C-index** grades the ranking; **Brier** grades the probabilities.

10 One-Page Cheat Sheet

Term	What it means in one line
Event	The thing you are timing (death, failure, churn).
Censoring	“Still alive when we stopped watching” — known partly, not fully. The reason this field exists.
Right-censored	Most common: event has not happened yet at the cutoff.
Left truncation	Subject only enters the study late; you could not have seen earlier failures.
$S(t)$ survival	% still alive at time t (the scoreboard; only goes down).
$h(t)$ hazard	Risk rate <i>right now</i> , for survivors only (the danger meter).
Kaplan–Meier	The downward staircase; steps at churns, censored people leave quietly.
Median survival	Where the staircase crosses 50% (“typical” lifespan); may be “not reached”.
Log-rank test	Do two groups’ curves really differ? Gives a yes/no (p-value), not a size.
Cox model	Each trait <i>multiplies</i> a shared baseline risk.
Hazard ratio (HR)	The multiplier: > 1 riskier, < 1 protective, $= 1$ no effect.
PH assumption	Cox assumes the multiplier stays constant over time; check with parallel-curve / Schoenfeld tests.
Weibull shape k	$k > 1$ risk rises, $k = 1$ flat, $k < 1$ risk falls (early churn then loyal).
Parametric vs Cox	Use parametric to forecast the future / get smooth curves / read the shape.
RSF	Random forest for time-to-event; no PH assumption; finds interactions.
XGBoost AFT	Boosting that predicts the actual time; censored rows get a time <i>range</i> .
C-index	“Did it rank who-fails-first correctly?” = survival AUC (0.5 random, 1.0 perfect).
Brier score	“Are the predicted probabilities accurate?” Lower is better.
lifelines	Friendly library: KM, Cox, log-rank, plots.
scikit-survival	sklearn-style: RSF, C-index, Brier.

11 Likely Questions (with plain answers)

1. *Why not just use normal regression?* — Because of censored people: deleting them biases you, and pretending they churned at the cutoff is a lie. Survival math uses them correctly.
2. *Hazard vs probability?* — Hazard is a *rate* for those still alive right now; it can be bigger than 1 and is not itself a probability.
3. *Why doesn’t the KM curve drop when someone is censored?* — They never churned; they just left the pool, lowering the “at-risk” count for later steps.
4. *What does $HR = 0.7$ mean?* — 30% *lower* churn rate at every time point (protective) — assuming the proportional-hazards assumption holds.
5. *How do you check that assumption, and what if it fails?* — Parallel-curve (log–log) plot or Schoenfeld test; fix by stratifying, letting the effect vary with time, or using Weibull / RSF.
6. *When is Weibull better than Cox?* — When you need to forecast beyond your data, want smooth curves, or the rising/flat/falling *shape* is itself the answer.

7. *What does $k < 1$ tell a business?* — Risk falls over time: heavy early churn, then survivors stay loyal $\Rightarrow$ invest in onboarding.
8. *C-index vs Brier?* — C-index = ranking (AUC-like); Brier = are the probabilities accurate. Report both.
9. *Why did your team use classification, not survival, for churn?* — The label is a fixed-window yes/no, so classification directly optimises the scored metric; survival would be for predicting churn *timing*.